

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CSA1610

Student Name:-D.Deepak

COURSE CODE: Data Warehousing and Data Mining

REG NO:-192572095

COURSE OUTCOME COVERED

CO4: Examine the applicability of clustering algorithms in real-world scenarios, presenting findings through clear reports with effective visualizations.

QUESTIONS: 05

Total Marks:50

Annexure A

Data Interpretation

Code of Conduct:

I **D.Deepak/[192572095]** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:*deepak*

Q1. Customer Data — K-Means vs Hierarchical Clustering

Dataset:

Customer	Income (₹)	Spending Score
C1	30,000	60
C2	60,000	40
C3	50,000	70
C4	28,000	65
C5	65,000	35

Answer

K-Means and Hierarchical clustering can both be used to group customers according to their income and spending behavior.

In K-Means, the number of clusters is decided before clustering. In this dataset, customers with similar characteristics are grouped together. C1 and C4 have relatively low income and high spending scores, while C2 and C5 have high income and low spending scores. C3 has high income and a high spending score, so it differs from the other groups.

Criteria

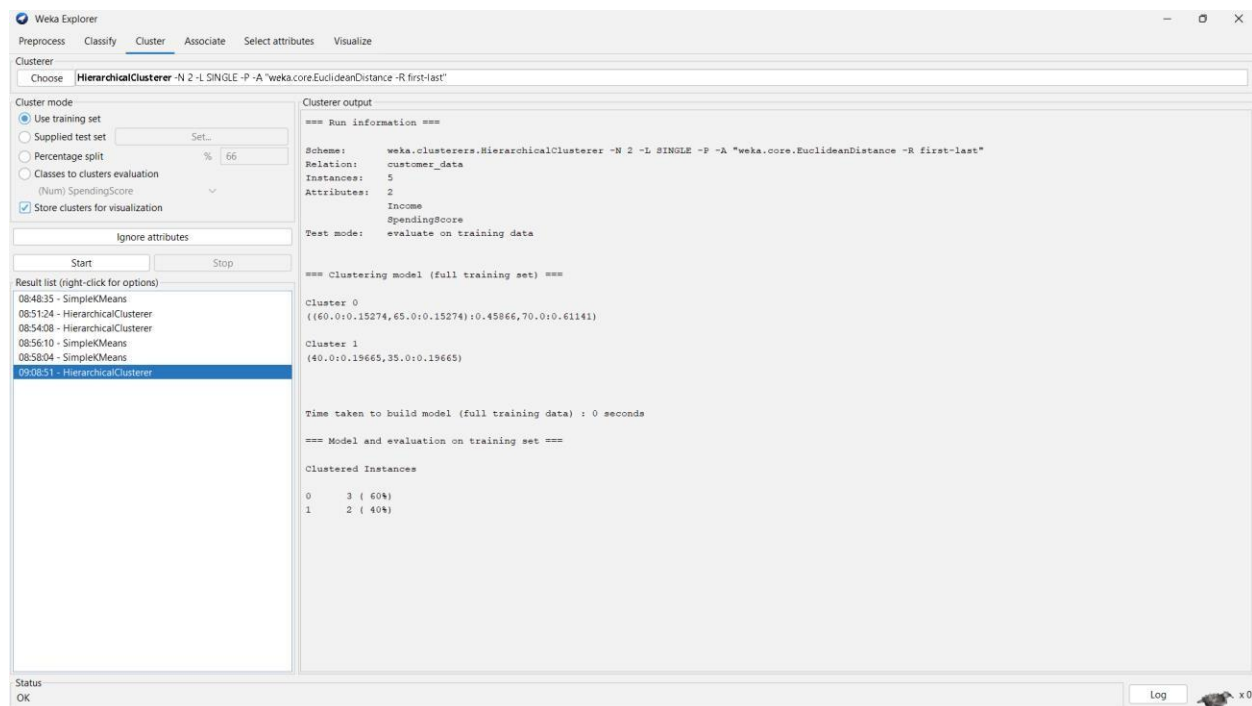
K-Means

Hierarchical



Approach	Partition-based	Tree-based
Number of clusters	Specified in advance	Can be selected from dendrogram
Speed	Faster	Slower
Scalability	Good for large datasets	Better for smaller datasets
Visualization	Cluster plot	Dendrogram

Conclusion: K-Means is generally more suitable for large customer datasets because it is faster and more scalable. Hierarchical clustering is useful for smaller datasets where understanding relationships between customers is important.



Q2. Document Data — K-Means vs EM

Dataset:

Document	Word1	Word2	Word3
D1	0.2	0.5	0.3
D2	0.1	0.6	0.3
D3	0.8	0.1	0.1
D4	0.75	0.15	0.1
D5	0.2	0.4	0.4

Answer

K-Means assigns each document to one specific cluster based on its distance from the cluster centroid. It works well when the clusters are clearly separated.

Criteria	K-Means	EM
Assignment	Hard assignment	Probabilistic assignment
Overlapping clusters	Less suitable	More suitable
Output	One cluster per document	Probability for each cluster
Complexity	Simpler	More complex
Best use	Clearly separated data	Overlapping data

Conclusion: For overlapping document clusters, EM is more flexible than K-Means because it represents the probability of a document belonging to different clusters.

The screenshot shows the Weka Explorer interface with the 'Cluster' tab selected. The 'SimpleKMeans' algorithm is chosen with the following parameters: -init 0 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -N 2 -A "weka.core.EuclideanDistance -R first-last" -I 500 -num-slots 1 -S 10. The 'Cluster mode' section has 'Use training set' selected. The 'Clusterer output' pane displays the following results:

```
=== Clustering model (full training set) ===

kMeans
=====

Number of iterations: 2
Within cluster sum of squared errors: 0.17523053665910012

Initial starting points (random):

Cluster 0: 0.75,0.15,0.1
Cluster 1: 0.1,0.6,0.3

Missing values globally replaced with mean/mode

Final cluster centroids:

Attribute      Full Data      Cluster#
              (5.0)         (2.0)         (3.0)
=====
Word1           0.41           0.775         0.1667
Word2           0.35           0.125         0.5
Word3           0.24           0.1           0.3333

Time taken to build model (full training data) : 0 seconds

=== Model and evaluation on training set ===

Clustered Instances

0      2 ( 40%)
1      3 ( 60%)
```

Q3. Geographic Data — Euclidean vs Manhattan Distance

Dataset:

Point	Latitude	Longitude
P1	13.08	80.27
P2	13.10	80.25

P3	12.97	80.22
P4	13.05	80.30
P5	13.00	80.20

Answer

Euclidean distance measures the straight-line distance between two points.

Formula:

$$\text{Euclidean Distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Manhattan distance measures distance by adding the absolute differences between the coordinates.

Formula:

$$\text{Manhattan Distance} = |x_2 - x_1| + |y_2 - y_1|$$

For example, between P1 and P2:

Euclidean:

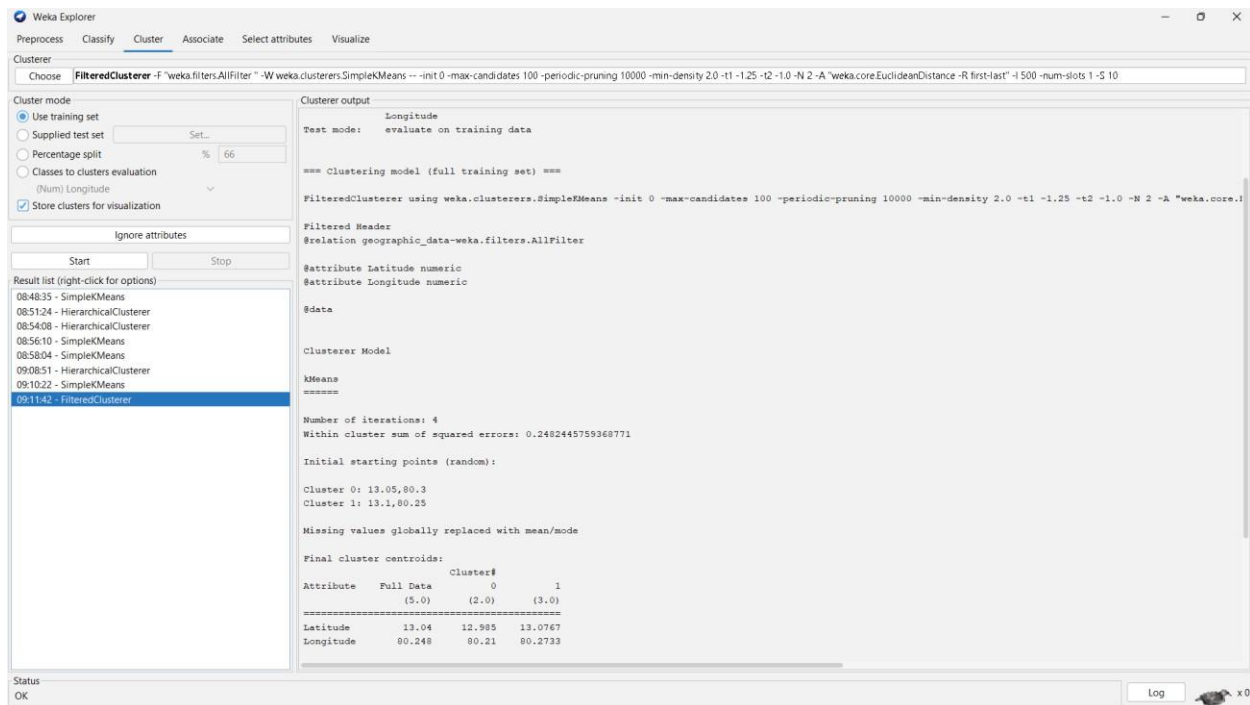
$$\begin{aligned} & \sqrt{((13.10 - 13.08)^2 + (80.25 - 80.27)^2)} \\ &= \sqrt{(0.0004 + 0.0004)} \\ &\approx \mathbf{0.0283} \end{aligned}$$

Manhattan:

$$\begin{aligned} & |13.10 - 13.08| + |80.25 - 80.27| \\ &= 0.02 + 0.02 \\ &= \mathbf{0.04} \end{aligned}$$

Feature	Euclidean	Manhattan
Distance	Straight-line	Coordinate-wise
Formula	Square root of squared differences	Sum of absolute differences
Sensitivity	More affected by large differences	Less affected by individual large differences
Suitable for	Geometric/continuous data	Grid-like or coordinate-based movement

Conclusion: Both distance measures can produce different distance values and may therefore affect cluster formation. Euclidean distance is suitable when direct geometric distance is important, while Manhattan distance can be useful when movement is considered along separate coordinate directions.



Q4. Retail Products — Agglomerative vs Divisive Clustering

Dataset:

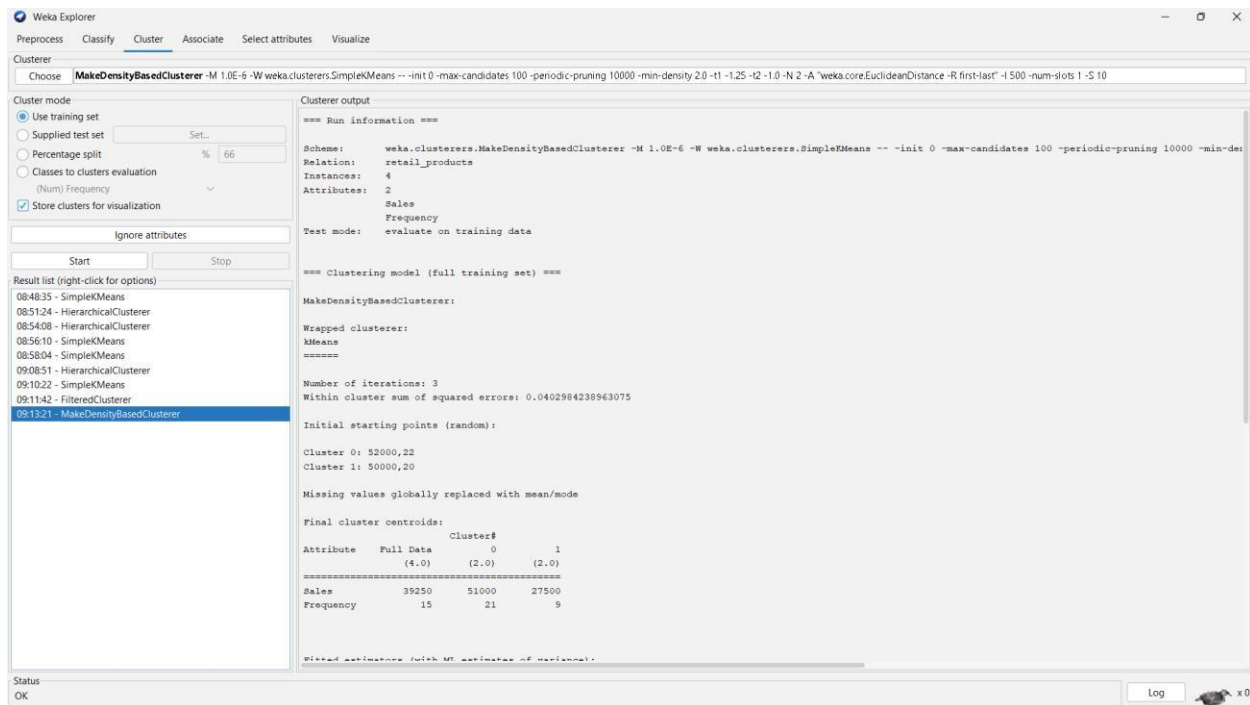
Product	Sales (₹)	Frequency
P1	50,000	20
P2	30,000	10
P3	52,000	22
P4	25,000	8
P5	32,000	12

Answer

Agglomerative clustering is a bottom-up approach. Initially, every product is treated as an individual cluster. The most similar products are then combined step by step until the required groups are formed.

Divisive clustering is the opposite approach. It starts with all products in one large cluster and repeatedly divides the cluster into smaller groups.

Conclusion: For this small retail dataset, agglomerative clustering is a practical choice because it provides an easy way to identify similar products based on sales and purchase frequency.



Q5. Mixed Attributes — Centroid-Based vs Density-Based Clustering

Dataset:

Item Price (₹) Category Rating

I1	500	A	4.5
I2	300	B	3.8
I3	550	A	4.7
I4	250	B	3.5
I5	520	A	4.6

Answer

Centroid-based clustering, such as K-Means, forms clusters around a central point called a centroid. It works well with numerical attributes such as price and rating, but the categorical attribute Category cannot be directly handled by standard K-Means without suitable encoding.

Criteria	Centroid-Based	Density-Based
Example	K-Means	DBSCAN
Main idea	Groups around centroids	Groups based on density

Numerical data	Very suitable	Suitable
Categorical data	Requires encoding/adaptation	Requires suitable distance handling
Outlier detection	Limited	Strong
Cluster shape	Usually compact	Can detect irregular shapes

Interpretation

For this dataset, the numerical attributes show clear groups, but the Category attribute cannot simply be treated as a normal numerical value. Therefore, preprocessing or an appropriate mixed-data distance measure is needed before clustering.

Conclusion: Centroid-based clustering is effective when the data is mainly numerical and clusters are compact. Density-based clustering is useful when clusters may have irregular shapes or when outliers are important. For this mixed dataset, the categorical attribute must be handled carefully before applying either method.

The screenshot shows the Weka Explorer application window. The 'Clusterer' tab is active. In the 'Cluster mode' section, 'Use training set' is selected. The 'Ignore attributes' section is empty. The 'Result list' on the left shows various clustering algorithms, with '02:14:42 - Cobweb' selected. The 'Clusterer output' on the right displays the command used to run the Cobweb algorithm and its output, including the number of clusters (8) and the time taken to build the model (0.01 seconds).

```

=== Run information ===

Scheme:      weka.clusterers.Cobweb -A 1.0 -C 0.0028209479177387815 -S 42
Relation:    mixed_attributes
Instances:   5
Attributes:  3
              Price
              Category
              Rating
Test mode:   evaluate on training data

=== Clustering model (full training set) ===

Number of merges: 0
Number of splits: 0
Number of clusters: 8

node 0 [5]
| node 1 [3]
| | leaf 2 [1]
| node 1 [3]
| | leaf 3 [1]
| node 1 [3]
| | leaf 4 [1]
node 0 [5]
| node 5 [2]
| | leaf 4 [1]
| node 5 [2]
| | leaf 7 [1]

Time taken to build model (full training data) : 0.01 seconds

=== Model and evaluation on training set ===

Clustered Instances
2      1 ( 20%)

```